



Where the **agentic stack** is being built.

MAS-Lab:

An Open Framework for Spec-Driven
& Interoperable Multi-Agent Systems

Jordan Augé (Cisco Systems)

Who am I ?

- Tech lead, Outshift @ Cisco since 2014
- Mixed academic & Industrial background
 - Traffic engineering ; QoS modeling
 - Internet measurements & testbeds (PlanetLab team)
 - Information Centric Networking
 - Cloud Security & AI/ML
 - Agentic AI
- Outshift @ Cisco (<https://outshift.cisco.com/>)
 - Quantum Networking & Agentic AI
 - Internet of Agents (AGNTCY: <https://agntcy.org/>)
 - Internet of Cognition (more at the end of the talk)
- Chair of Accuracy and Reliability WG @ AAIF



Observe & Eval project
splunk>



- **Motivations**: testing and verifying agentic systems
- **MAS-Lab** introduction: specs, runtime, labs, demo
- Objective : **Accuracy & Reliability** for MultiAgent Systems
- Example: Investigating **Cognitive failures**

Motivation

Multi-Agent System + Network Digital Twin for network reliability

Replay real incidents on a replica of production networks

joint work with  **swisscom**

DEVELOPMENT

Rapid iterations to design
Versioning and benchmarking
Test new solutions

DEPLOYMENT

Our stack: llamaindex



MCP



A2A



OpenTelemetry

Logic & deployment mixed in code
Lack of specs
Breaking changes in frameworks

RESEARCH & LARGE-SCALE EVALUATION

Hard to control, scale & parallelize
Test logic only



- J. Augé, S.Betts, G.Carofiglio, G. Grassi, M. Gysi, JK d'Souza - **Aether: Network Validation Using Agentic AI and Digital Twin** – ArXiv preprint - <https://arxiv.org/abs/2604.18233>
- G.Carofiglio, JK d'Souza, M. Gysi, O. Mabadeje - **Driving autonomous network operations - How Swisscom and Outshift by Cisco are redefining network reliability with AI agents** – [Whitepaper](#), 2025

Where the **agentic stack** is being built.

MAS-Lab overview

MAS-Lab is a **declarative agentic framework** for composition, validation, experimentation and pre-production testing of agentic applications.

SPECS

Declares all system entities—agents, tools, models, and coordination edges in versioned YAML specifications. **Grounding for observability.**

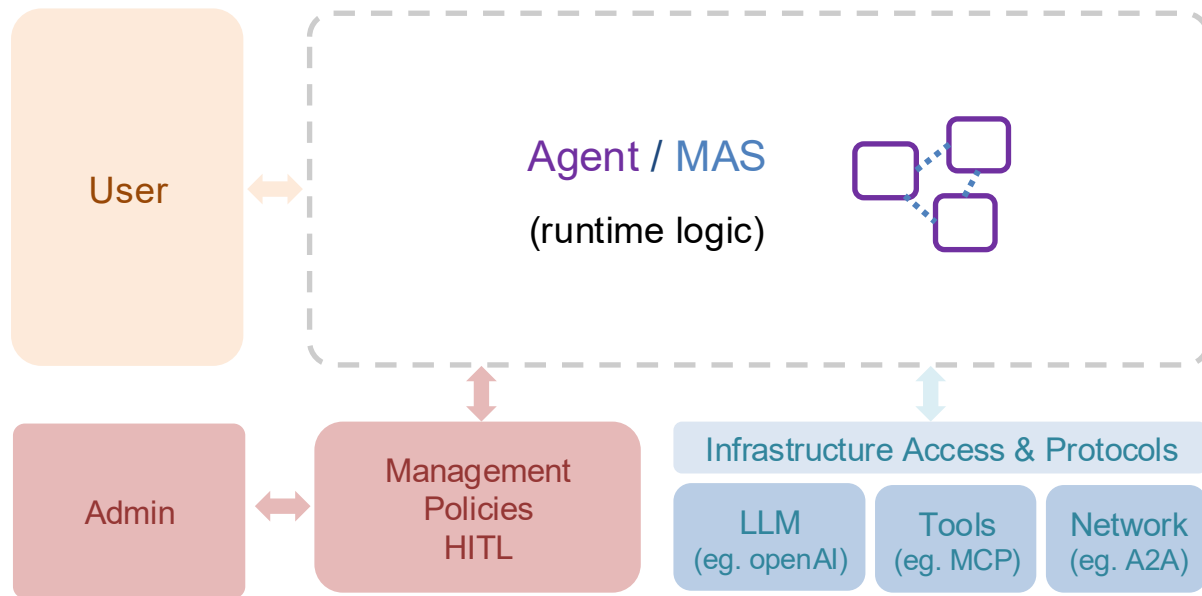
RUNTIME

A **modular & safe runtime** allowing implementation of specs (logic, control, infra) through plugins, on top of a stable API interface (contracts), with **observability and governance as first-class citizens.**

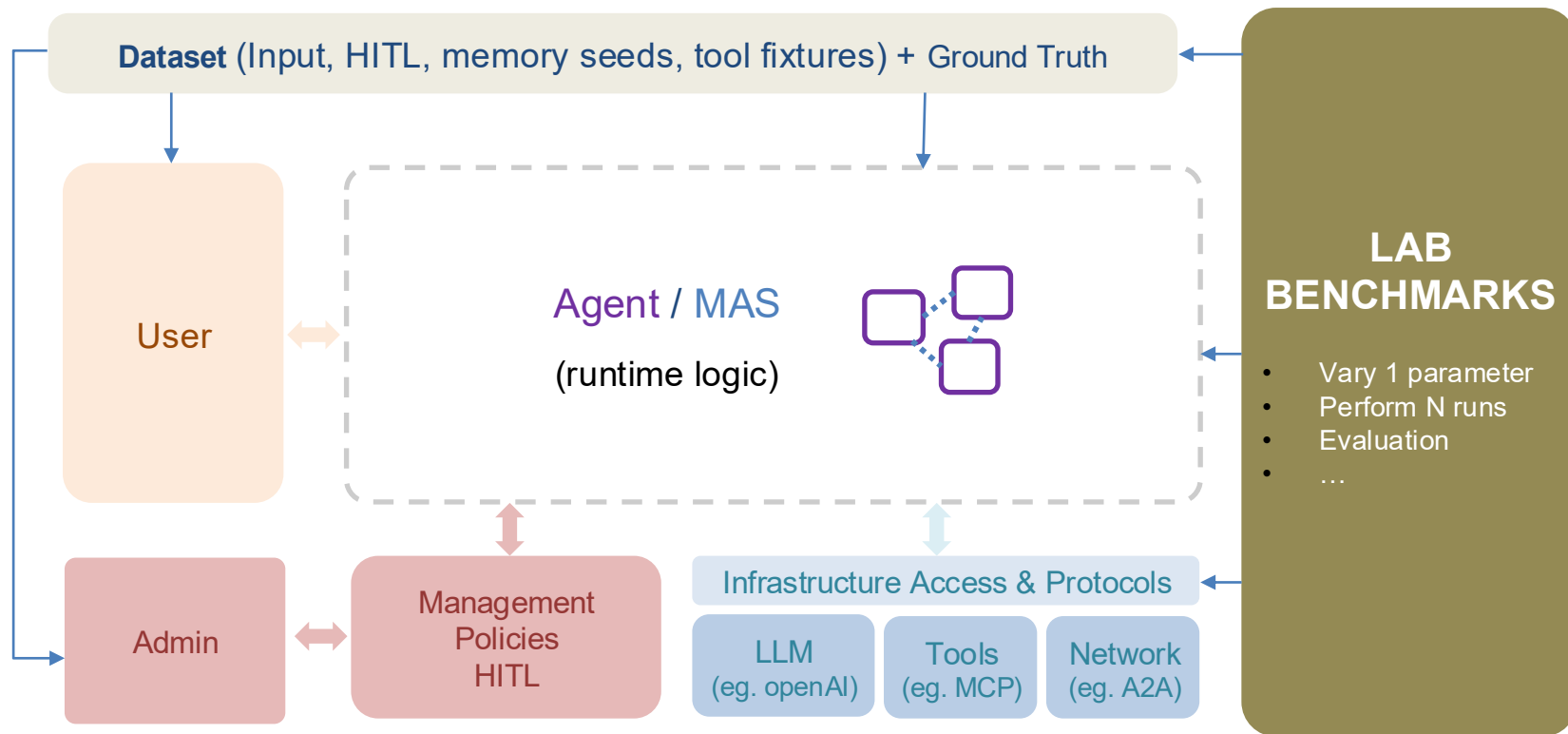
LAB

Allow system behaviour study and validation under **controlled variations** (explore design space, swap infrastructure, inject faults, replay traces)

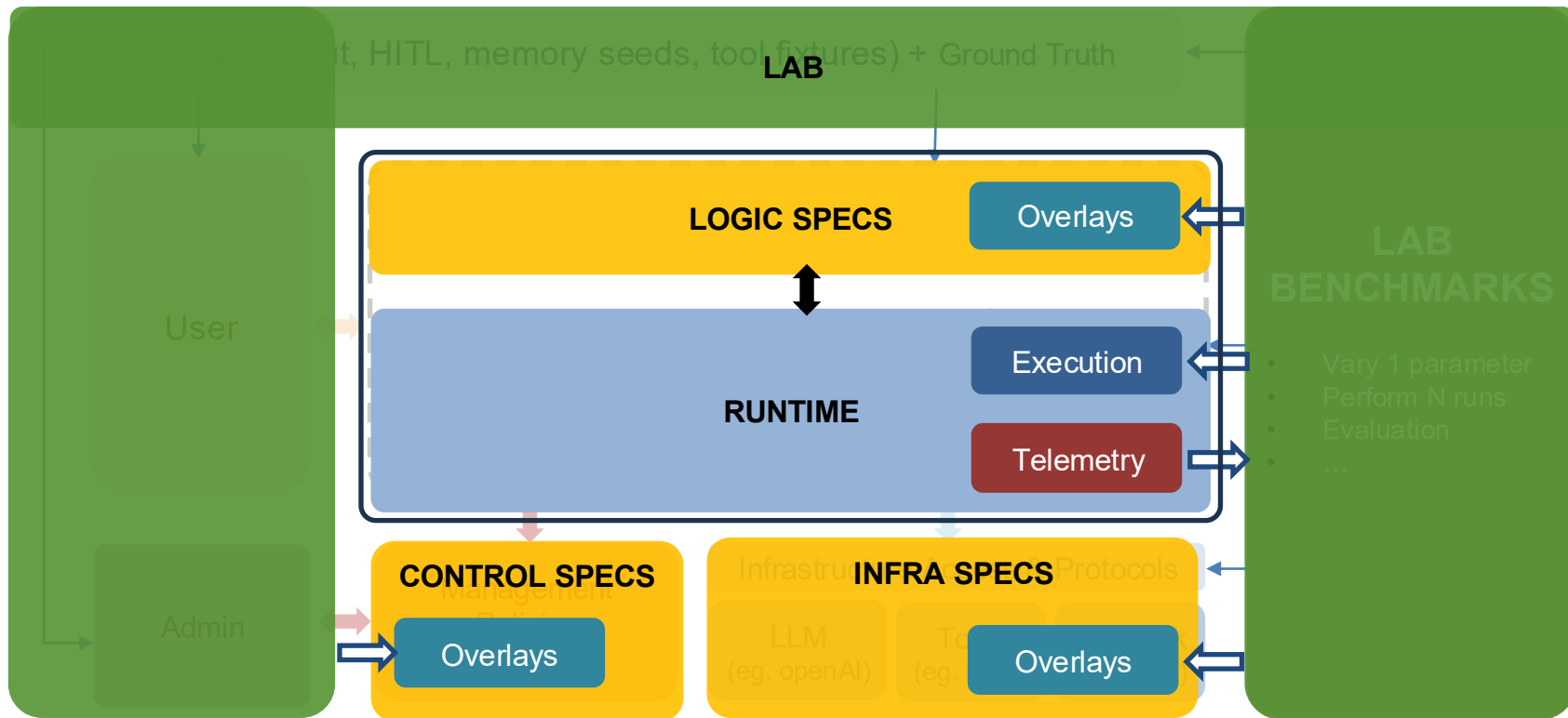
Agentic apps environment



Agentic apps environment

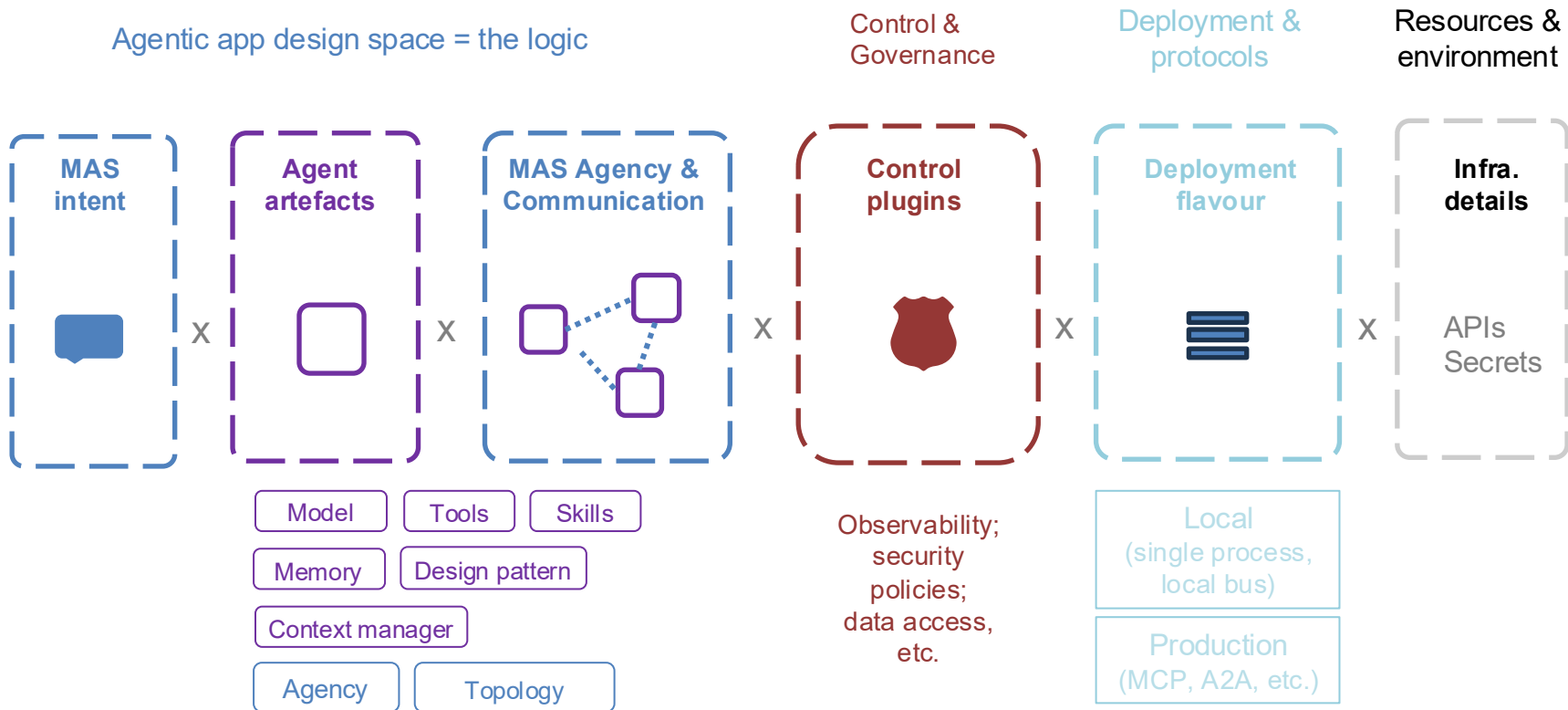


Agentic apps environment



MAS-Lab workflows

Agentic app design space = the logic



Where the **agentic stack** is being built.

DEMO

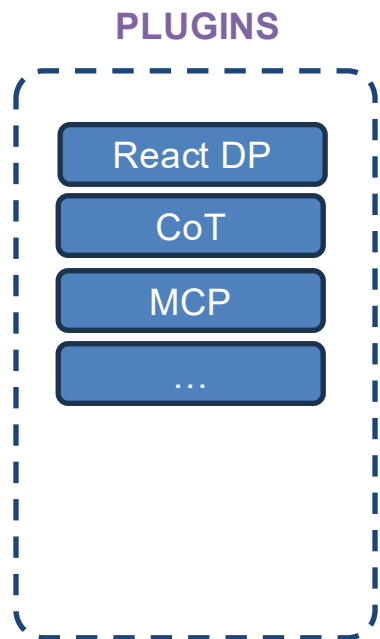
- MAS-Lab specs
- Agent & MAS creation
- Telemetry & plots
- Labs benchmarks



MAS-Lab tutorials

<https://outshift-open.github.io/mas-lab/tutorials/>

Runtime architecture

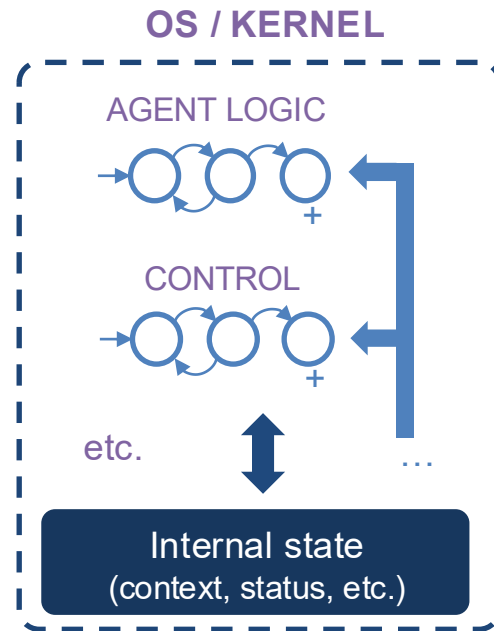


**COMPOSITION OF
ISOLATED PLUGINS**

Contrats / Protocols
(local, gRPC)



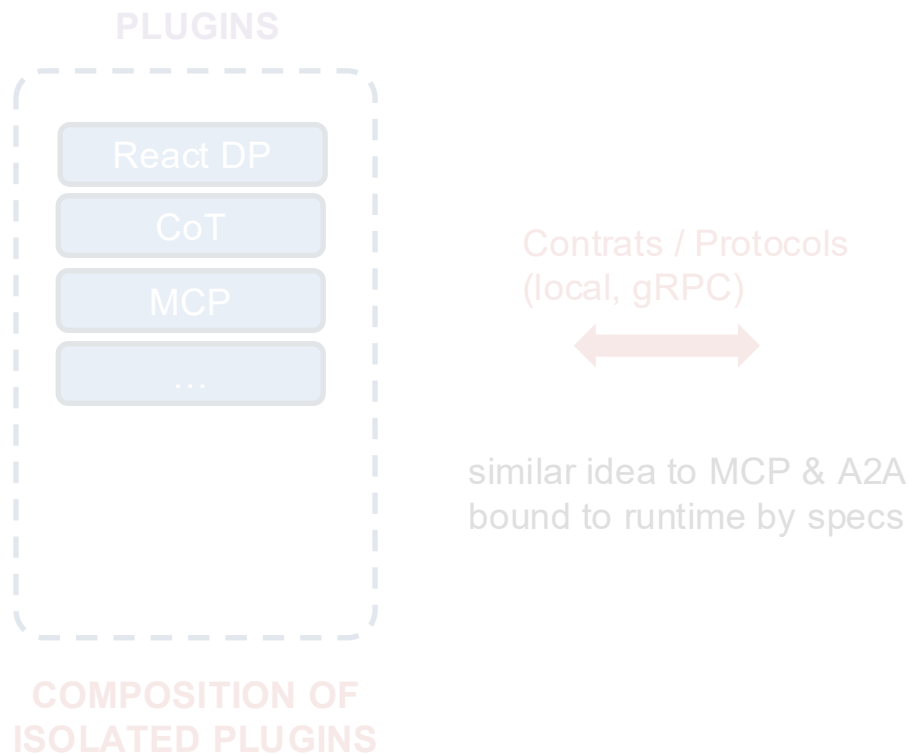
similar idea to MCP & A2A
bound to runtime by specs



COUPLED MEALY MACHINES

- ➔ proven behaviour (TLA+)
- ➔ natural MAS extension with Petri Nets

Runtime architecture



Verifiable runtime & MAS

- Plugins get **(restricted) access to resources** through contracts
- Enforced **governance & observability**
- **Safety** & proven behaviour: concurrency; deadlock- & loop-free, ...
- **Extensible** state machine (policies, etc.)
- python → rust : **memory safety**

Runtime architecture

Verifiable plugins

- Tested in isolation
- Invariants
- Unit and functional tests
- Gated access to runtime resources

Contracts / Protocols
(local, gRPC)



similar idea to MCP & A2A
bound to runtime by specs

Verifiable runtime & MAS

- Plugins get **(restricted) access to resources** through contracts
- Enforced **governance & observability**
- **Safety** & proven behaviour: concurrency; deadlock- & loop-free, ...
- **Extensible** state machine (policies, etc.)
- python → rust : **memory safety**

Runtime architecture

Verifiable plugins

- Tested in isolation
- Invariants
- Unit and functional tests
- Gated access to runtime resources

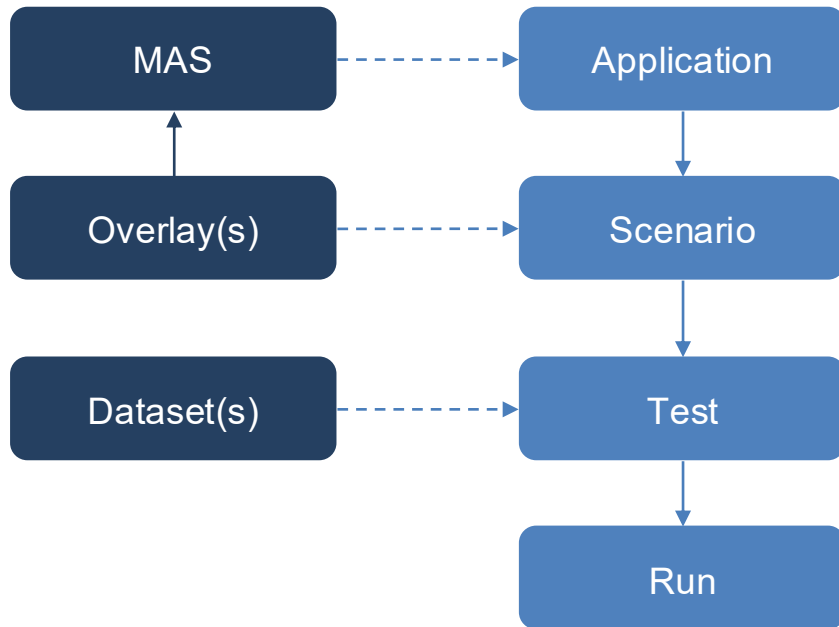
Verifiable specs

- Spec can be validated before execution
- Anchoring for observability
- Basis for enforcement

Verifiable runtime & MAS

- Plugins get **(restricted) access to resources** through contracts
- Enforced **governance & observability**
- **Safety** & proven behaviour: concurrency; deadlock- & loop-free, ...
- **Extensible** state machine (policies, etc.)
- python → rust : **memory safety**

Labs: designing experiments



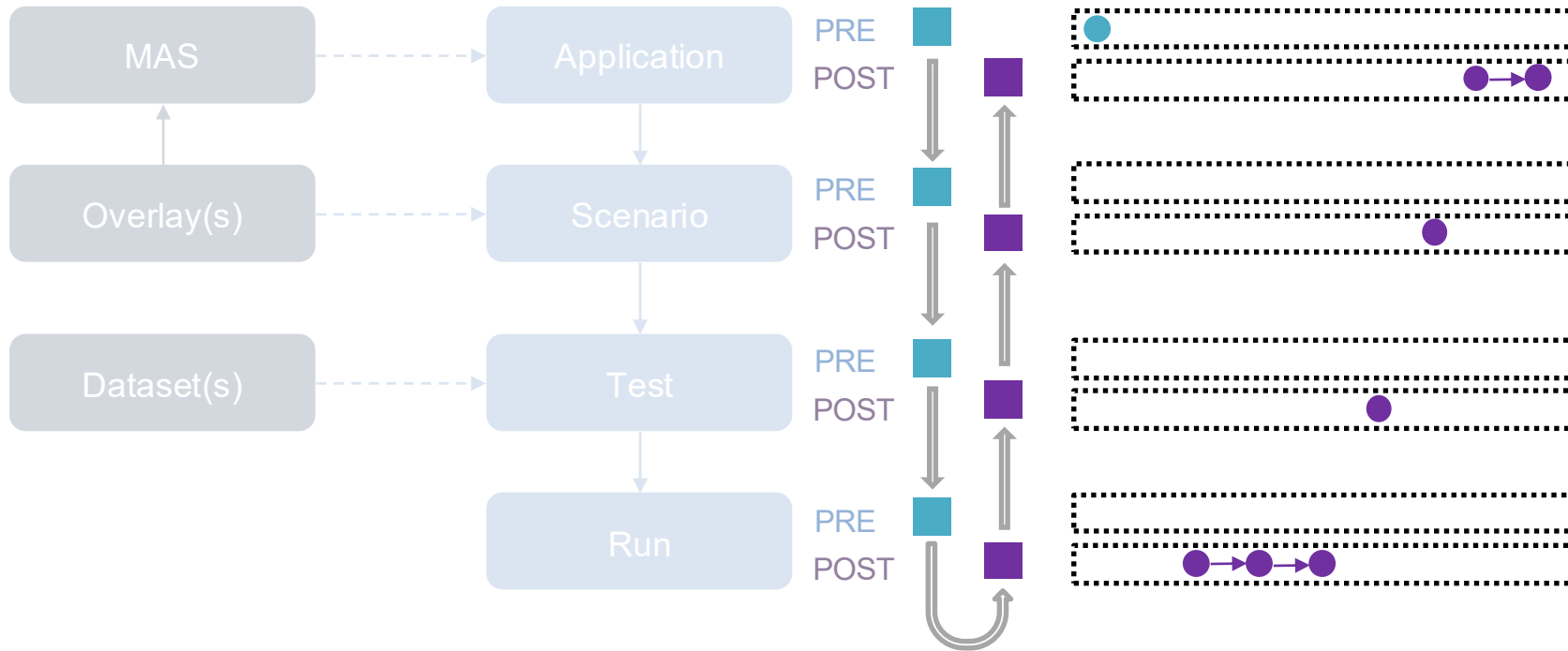
Baseline application(s) configuration

Variations through overlays
(logic / control / infrastructure)

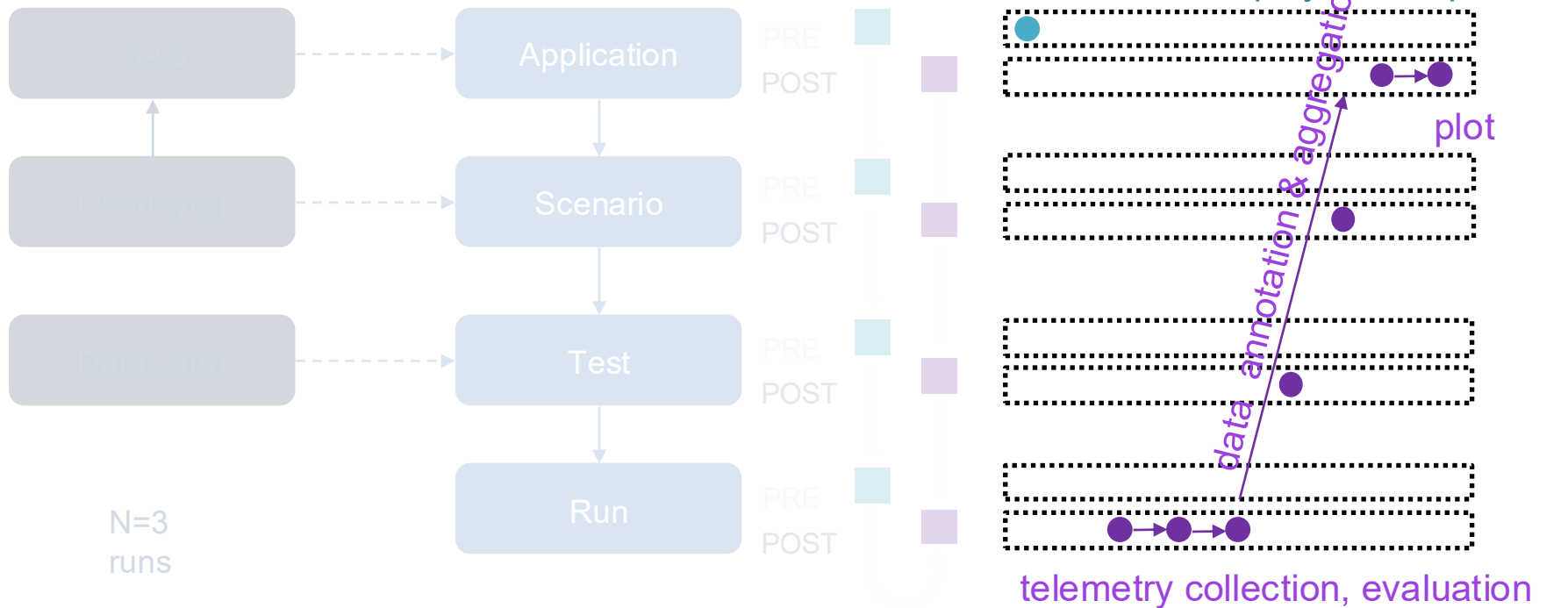
Iterate over dataset
(user prompt, test fixtures, memory seeds, ...)

Repeat N times
(statistical confidence)

Labs: designing experiments



Labs: designing experiments



Brownfield experiments

- **The Native Architecture**

- MAS-Lab ships out-of-the-box powered entirely by its own native runtime for optimal performance.
- *Example:* Building a greenfield agent application directly designed for the native execution engine.



- **Bridging External Frameworks**

- Integrate existing apps (brownfield projects) by writing custom adapters or wrapping third-party frameworks.
- *Example:* Wrapping an existing LangGraph app to inject external LLM/MCP proxies and collect OpenTelemetry.

- **Managing Trade-offs & Migration**

- Accept limitations on native flexibility and telemetry control in exchange for interoperability.
- *Example:* Use these wrappers temporarily to prove functional equivalence, establish a baseline, and guide your migration to a native MAS-Lab setup.

DEMO

- Labs benchmarks



MAS-Lab tutorials

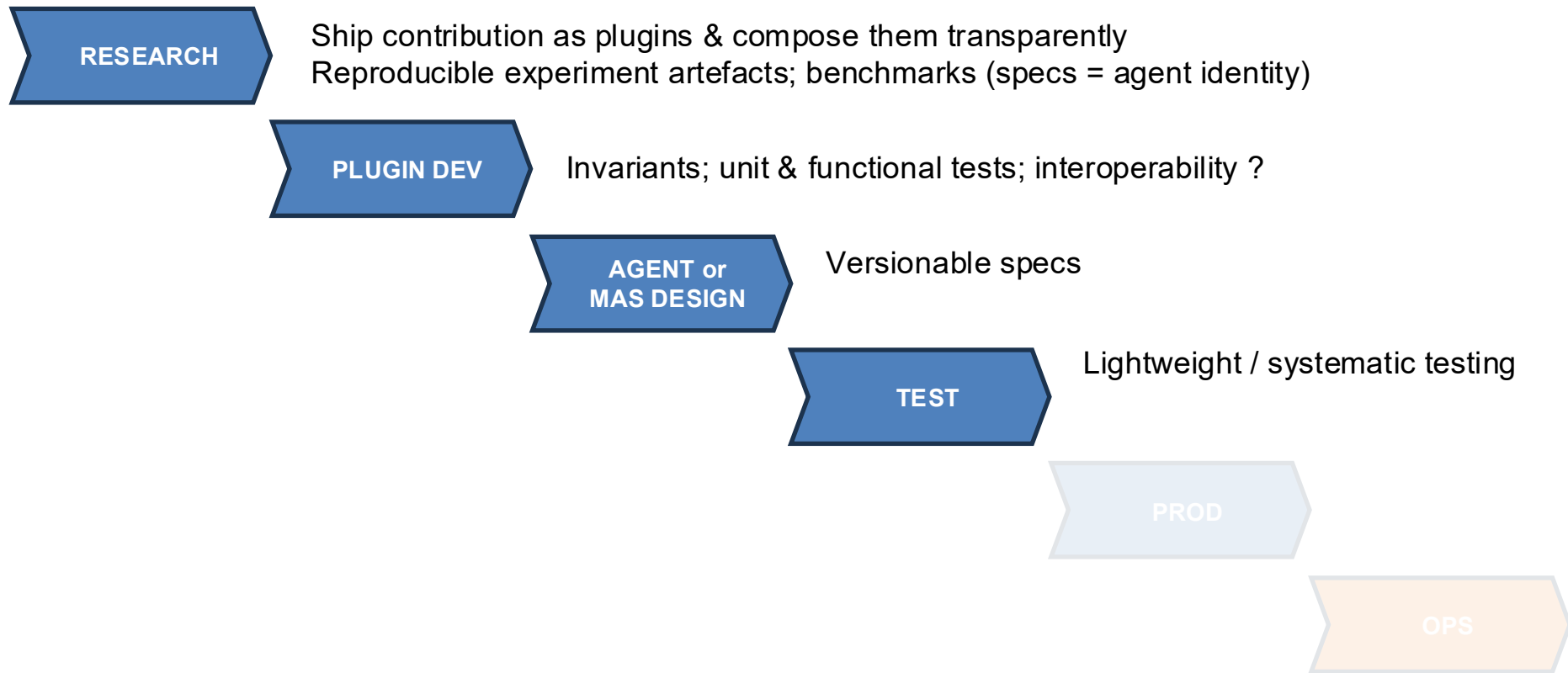
<https://outshift-open.github.io/mas-lab/tutorials/>

Reliability across the experiment lifecycle

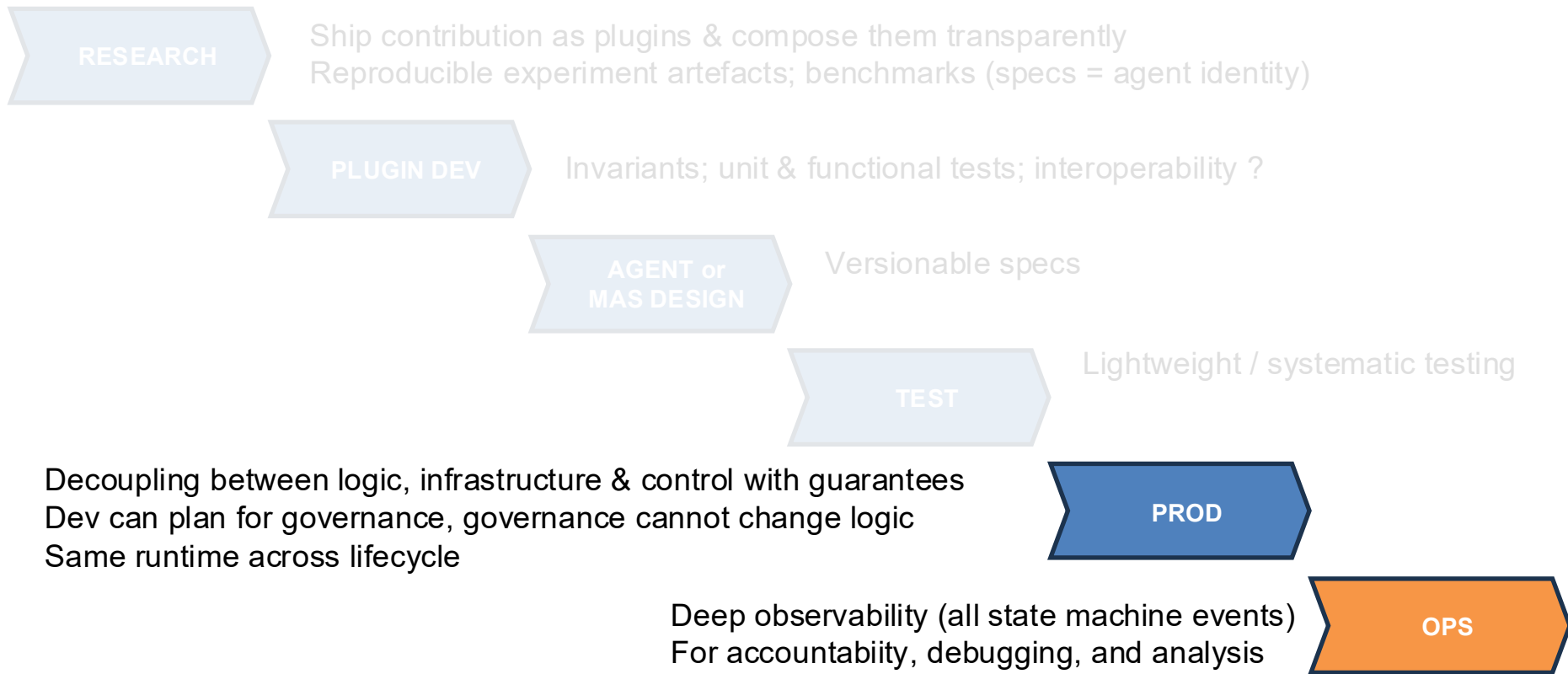
Handling cognitive failures

Where the **agentic stack** is being built.

Application lifecycle



Application lifecycle



Cognitive failures

Typical failures **show up in logs**



Repeated steps; loops



Handoff or network error



Tool call error; wrong parameter

Cognitive failures are **silent**: no error, plausible action...
... but wrong reasons



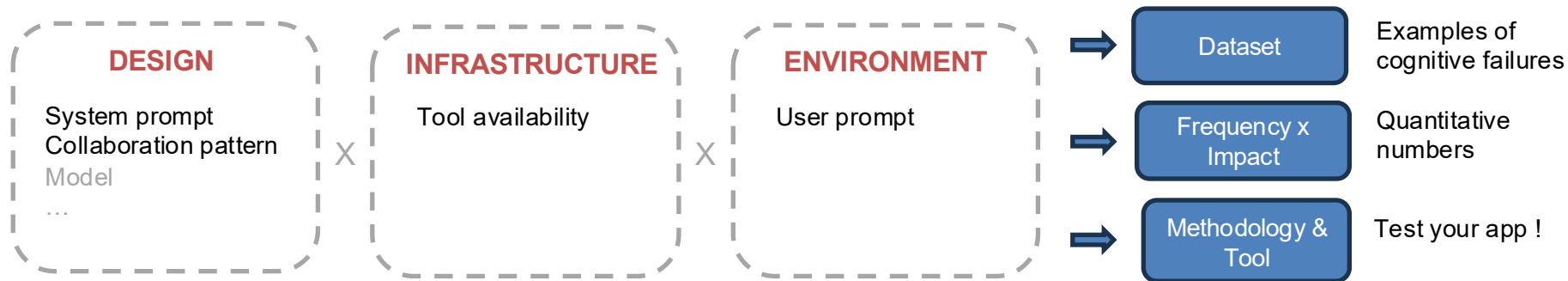
Lack of verification



Convergence issues



Ungrounded claims



Paper (under submission)

Blog: Silent Failures, Loud Consequences: Cognitive Challenges in Multi-Agent Systems (paper under submission)

<https://outshift.cisco.com/blog/ai-ml/cognitive-challenges-in-multi-agent-systems>

Cognitive failures

Typical failures **show up in logs**



Repeated steps; loops



Handoff or network error



Cognitive failures are **silent**: no error, plausible action...

... but wrong reasons



Lack of verification



Convergence issues

High prevalence of cognitive failures !

- common test applications
- popular agentic apps (eg. ChatDev)
- even for moderator-based patterns (not only distributed)

les of
ve failures

ative
rs

Test your app !

Methodology &
Tool

System
Collabor
Model
...

Paper (under submission)

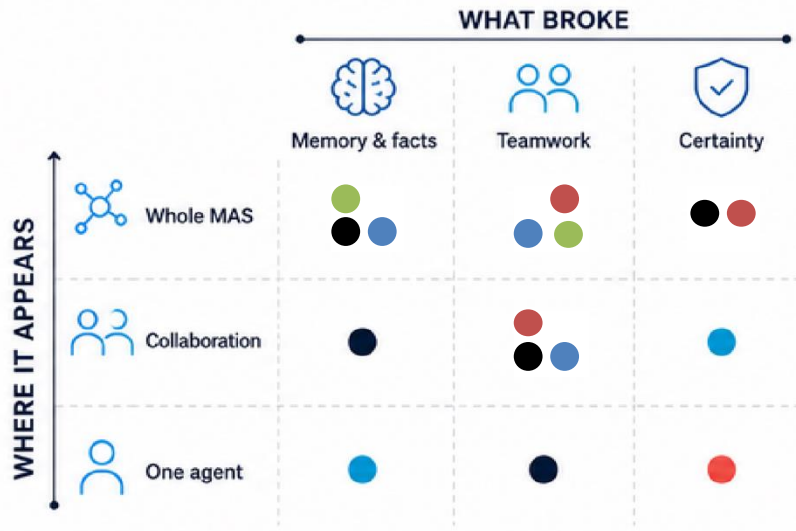
Blog: Silent Failures, Loud Consequences: Cognitive Challenges in Multi-Agent Systems (paper under submission)

<https://outshift.cisco.com/blog/ai-ml/cognitive-challenges-in-multi-agent-systems>

Cognitive failures taxonomy

TAXONOMY MATRIX

Extends the
MAST taxonomy



Different symptoms & root causes -> targeted remediation

(Deep) Observability & Semantic Analysis

DETECT

CLASSIFY

MITIGATE

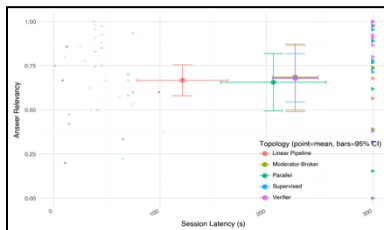
Paper results & next steps



Agentic developer (specs & logic)

Agent & MAS design space exploration

- Vary design pattern & topology
- Analysis pipeline with evaluation (Agntcy MCE)
- Design tradeoffs (metrics vs overhead)

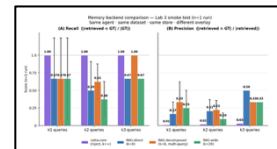


Researcher (lab, plugins & telemetry)

Extensions and reproducible research

Memory benchmark:

- Wrap OSS implementation
- Insert through overlay
- Leverage native observability for performance analysis



Enterprise operator (control & governance)

Lifecycle control : From dev to production



MAS-Lab

<https://github.com/outshift-open/mas-lab>

A **specification-driven foundation** for building multi-agent systems that are testable, reproducible, observable, and governable from design to production.

License `Apache 2.0` python `3.11+`

v0.1

Initial release

Agent runtime ; logic specs.; control plugins; labs/benchmark

<https://arxiv.org/abs/2606.30546>

v0.2

(end Sept'26)

- AgentSkills.io; MCP; A2A; Otel; ...
- Collaboration patterns

Call to action

MAS-Lab – Code, doc, blog, labs & tutorials

We welcome feedback and collaborations to shape evolution of the tool, libraries and use cases.
We want to make it a useful for the community.

<https://github.com/outshift-open/mas-lab>



Blog : Silent Failures, Loud Consequences: Cognitive Challenges in Multi-Agent Systems

<https://outshift.cisco.com/blog/ai-ml/cognitive-challenges-in-multi-agent-systems>



We'll have a hands on workshop soon !

Silent Failures, loud consequences: hands-on diagnosis & remediation in Multi-Agent Systems with MAS-Lab & OXP

Virtual event – Oct 27th, 2026 - **Shift.forward()** - You can register !



Help progress the work on Accuracy and Reliability, join us !
Contribute to the Accuracy and Reliability WG Survey



Agentic AI Foundation

AGNTCon + MCPCon Europe

Where the **agentic stack** is being built.